NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

AY2025/2026 Semester 2

SC4023 Big Data Management

Group 24

Semester Group Project

Contribution Form

Group 24

Name	Matriculation Number	Detailed Individual Contribution	Percentage
Isaac Lim Jia Le	U2321202C	Data Processing layer:The way d the program scans data and finds values efficiently	33.3%
Maison Sapol	U2220031B	Data storage layer: Column store layout, encoding schemes, zone maps and index	33.3%
Boonkitrungpaisarn Potala	U2222312E	Experiment Result: Program execution and output verification	33.3%

<u>Name and Signature from all group members:</u>

Name and Signature of Member 1

Isaac Lim Jia Le

Name and Signature of Member 2

Maison Sapol

Name and Signature of Member 3

Boonkitrungpaisarn Potala

Data Storage

This section describes how the program stores the data using a column store. The CSV is first read into memory, then rows are sorted to cluster similar values together. Each row is encoded into compact binary representations and written into separate column files in fixed-size pages. During writing, zone maps and a month index are built to support page skipping at query time.

1.1 Column-Store Layout

The data is stored in a column-store layout where each of the 10 attributes occupies its own binary file on disk. For example, all month values are stored contiguously in _month, all town values in _town, and so on. This decomposition allows reading only the relevant columns, avoiding the I/O cost of loading unused attributes.
Each column file is divided into fixed-size pages of 4096 bytes (4 KB). Encoded values are packed sequentially into pages. The number of items per page depends on the column's byte width. For instance, the month column uses 2 bytes per value, fitting 2048 items per page, while the town column uses 1 byte per value, fitting 4096 items per page. When the next encoded value would not fit within the remaining space of the current page, the buffer is flushed to disk, padded with null bytes to fill the page to exactly 4096 bytes.

1.2 Encoding Schemes

Each column uses a type-specific encoder to convert raw CSV strings into fixed-width binary representations. We implement two store configurations for comparison, including a **basic** store using fixed-width ASCII strings, and a **compressed** store using typed encoders. The table below summarizes encoding and the resulting byte width.

Column	Basic (bytes)	Compressed (bytes)	Encoder
month	7	2	MonthEncoder (year*12 + month-1)
town	15	1	CategoricalEncoder (26 towns)
flat_type	16	1	CategoricalEncoder (7 types)
block	5	3	BlockIdEncoder (letter|number)
street_name	20	2	Dictionary encoding (577 streets)
storey_range	12	1	CategoricalEncoder (17 ranges)
floor_area_sqm	4	4	Float32
flat_model	22	1	CategoricalEncoder (21 models)
lease_commence_date	2	2	UInt16
resale_price	4	4	Float32
Total per record	117	21	

MonthEncoder maps the "YYYY-MM" date string to a single integer computed as year * 12 + (month - 1). This yields a unique 2-byte value per calendar month. This enables efficient range comparisons using simple integer arithmetic rather than string parsing.
CategoricalEncoder maps each known string value (e.g. the 26 town names) to a zero-based integer index. The byte width is the minimum number of bytes needed to represent all indices. For N categories, we need ceil(bit_length(N) / 8) byte. Since all categorical columns in this dataset have fewer than 256 categories, each requires only 1 byte.
BlockIdEncoder packs block identifiers like "123" or "123A" into 3 bytes. The letter (if present) is stored in the upper byte as its ASCII code, and the numeric portion occupies the lower 2 bytes as an unsigned 16-bit integer.
StreetNameEncoder uses dictionary encoding. The program first scans the CSV to extract all unique street names, sorts them, and assigns each a sequential integer index. The dataset contains 577 unique street names, so the encoder uses 2 bytes per value, replacing the naive 20-byte fixed-width string.

Overall, the compressed store reduces per-record storage from 117 bytes to 21 bytes, a compression ratio of roughly 5.6x. This reduces page I/O during queries because more records fit into each page.

1.3 Zone Maps and Month Index

Before writing records into the column store, the rows are sorted by (month, town, floor_area_sqm). It groups records with similar attribute values into the same pages, which improves the effectiveness of the zone maps described below.
Without sorting, a single page might contain records spanning many different months and towns. With sorting, pages tend to contain records from only one or a few months and towns, allowing the scan to skip entire pages that do not match the query's month range or target town.

Three additional data structures are built during store construction to allow skipping irrelevant pages without reading the column data.
Town Zone Map: For each page of the town column, a bitmask is maintained where bit *i* is set if town *i* appears in that page. During record insertion, the bitmask is updated using bitwise OR: *bitmask = bitmask OR (1 << town_index)*. Bitwise OR adds the new town's bit to the bitmask while preserving all previously set bits, so the final bitmask reflects every town that appears in that page. At query time, if the target town's bit is not set in a page's bitmask, that page is skipped.
Area Zone Map: For each page of the floor_area_sqm column, the minimum and maximum area values are tracked. At query time, if a page's maximum area is below the required minimum, the page is skipped. The reverse check (whether a page's minimum area exceeds some upper bound) is not needed because the query condition is area >= y with no upper limit on area.
Month Page Index: An array of sets is maintained, one per calendar month in the dataset's range. Each set stores the page numbers that contain at least one record with that month value. At query time, only pages appearing in the relevant month's set are scanned.
These structures are maintained in memory and are populated incrementally as records are inserted during store construction. During query time, the scan engine consults these structures to determine which pages to skip or read from disk.

1.4 Reading and Writing

Writing: Each row is fed to the column store one at a time. Every field is encoded and appended to a per-column write buffer. When the next value would cause the buffer to exceed 4096 bytes, the buffer is flushed to disk as a complete page.
Reading: During query processing, each column maintains a single-page read buffer. When a value at position pos is requested, the system computes which page it falls on. If that page is already in the buffer, the value is extracted directly. Otherwise, the page is loaded from disk. A read counter tracks total page I/O operations for performance measurement. A single-page buffer is sufficient since records are scanned in sequential order.
Output: The scan results are written to ScanResult_<MatricNum>.csv using csv.writer, following the output format specified in the requirements.

1.5 Exception Handling

Four columns are critical: month, town, floor_area_sqm, and resale_price. If any of these fields is empty in an input row, the row is rejected with a StorageError.
Encoding errors such as malformed date strings, unknown town names, or out-of-range block numbers are caught during the encode step and wrapped in a StorageError. In both cases, the failing row is skipped with a warning message rather than aborting the entire data load. This allows the store to be populated with all valid records even if a small number of input rows are malformed.
For query output, if no valid (x, y) pair produces a result with minimum price per square meter at most 4725, a single "No result" row is written to the output CSV.

Data Processing

2.1 Introduction

This section details the data processing mechanisms implemented in the SC4023 Project, focusing on how the program efficiently scans structured data, retrieves relevant records based on task-specific conditions, and optimizes performance through intelligent column scanning and result reuse. The processing pipeline is designed to handle moderate-to-large datasets while maintaining responsiveness and resource efficiency.

2.2 Column Scanning According to Task Conditions

The program implements conditional column scanning by adapting its filtering process to the requirements of each query. Instead of applying a fixed scan pattern for all cases, it evaluates task parameters such as the target month range, town, and minimum floor area, then scans only the relevant column data needed to satisfy those conditions.

This logic is handled through a set of filtering methods in the scan engine. The program first filters records by month, then by town, and finally by floor area. Each stage narrows the candidate positions before the next column is accessed, which helps reduce unnecessary reads. The filtering process can also take advantage of auxiliary structures such as the month index and zone maps to skip irrelevant pages before reading actual column values. (Figure 1 and Figure 2)

By combining condition-specific filters with storage-level optimizations, the system supports more efficient analytical queries on the column store. This makes the scanning process responsive to different task requirements while preserving the benefits of column-oriented access.

2.2.1 Vector-at-a-Time Column Scanning

The program implements a vector-at-a-time scanning strategy by processing the column store in fixed-size batches instead of scanning the entire dataset in a single pass. This approach improves efficiency by grouping records into manageable chunks and applying the same filtering conditions to each batch before computing aggregate results.

In the implementation, the vector_a_time() method divides the dataset into vectors of configurable size and evaluates each chunk separately. For every batch, the system reuses the existing column filters for month, town, and floor area, then computes statistics such as minimum price, average price, standard deviation, and minimum price per square meter. Partial results from each batch are combined to produce the final output. (Figure 3)

This reduces repeated full-dataset scans and supports more memory-efficient column processing. This makes the method a practical optimization for analytical workloads on the column store.

2.3.1 Retrieval Process

The retrieval process is designed around a column-oriented storage model, where each attribute is stored separately and accessed only when needed. This reduces unnecessary data reads and allows the system to evaluate query conditions directly on the relevant columns. The implementation combines column-level access with lightweight storage optimizations, including a page-level month index and zone maps for selected columns. The program begins in generate_scan_result(). (Figure 4)

The method iterates over all required task-condition pairs, defined by the month span x and minimum floor area y, and then evaluates each relevant town derived from the matric number. For every (x, y,

town) combination, the program invokes apply_filters() to identify the positions of records that satisfy the retrieval conditions. (Figure 2)

Within apply_filters(), the retrieval flow proceeds through a fixed sequence of column filters. First, filter_by_month() checks whether each record falls within the required month range. When enabled, this step consults the month index through pos_has_month_in_range() to skip irrelevant pages before accessing actual month values.

```
def pos_has_month_in_range(self, pos: int, month_start: int, month_end: int) -> bool:
    page_number = self._pos_to_page(pos, MONTH_COL)
    idx_start = month_start - MONTH_INDEX_OFFSET
    idx_end = month_end - MONTH_INDEX_OFFSET
    for month in range(idx_start, min(idx_end + 1, len(self.month_index))):
        if page_number in self.month_index[month]:
            return True
    return False
```

Next, filter_by_town() narrows the candidate set to records belonging to the required town, optionally using the town zone map from get_town_zone_map_entry(). Finally, filter_by_area() applies the minimum floor-area condition and can use the area zone map from get_area_zone_map_entry() to avoid scanning pages whose values cannot satisfy the threshold.

```
# Zone map / index accessors
def get_town_zone_map_entry(self, pos: int) -> int:
    return self._get_zone_map_entry(pos, TOWN_COL, self.town_zone_map)

def get_area_zone_map_entry(self, pos: int) -> tuple[float, float]:
    return self._get_zone_map_entry(pos, AREA_COL, self.area_zone_map)
```

After filtering, the resulting candidate positions are passed to find_min_price_per_sqm_record(), which identifies the qualifying record with the lowest resale price per square meter.

```
def find_min_price_per_sqm_record(self, positions: list[int]) -> tuple:
    """Returns (position, price_per_sqm) of the record with minimum price/sqm, or (None, inf)."""
    best_pos = None
    best_ppsqm = math.inf
    for pos in positions:
        price = self.store.get_resale_price(pos)
        sqm = self.store.get_floor_area_sqm(pos)
        ppsqm = price / sqm
        if ppsqm < best_ppsqm:
            best_ppsqm = ppsqm
            best_pos = pos
    return best_pos, best_ppsqm
```

If a valid record is found, get_record_details() retrieves the necessary output fields, such as year, month, town, block, floor area, flat model, and lease commencement date. (Figure 5).

At the storage layer, all record access is handled by DiskColumnStore. The method get_item() reads only the required page of the requested column and keeps that page in an in-memory buffer to reduce repeated disk reads. (Figure 6) This means the retrieval path benefits from column pruning, since only the columns involved in filtering or output are read, while unrelated columns remain untouched.

Experiment Result

This section shows the experiment results that the program successfully executes and produces correct outputs. The program was executed using matriculation number **U2222312E**, which corresponds to a query over the towns **CLEMENTI**, **CHOA CHU KANG**, and **BUKIT PANJANG** starting from

January 2022. To verify the correctness, the full output was validated against an independent Python reference implementation.

Program Execution

The program was executed using the following command:

python main.py ResalePricesSingapore.csv U2222312E

```
(sc4023) potalaboonkitrungpaisarn@Potalas-MacBook-Air SC4023-Project-main-main % python main.py ResalePricesSingapore.csv U2222312E
Matric: U2222312E
Year: 2022, Towns: ['CLEMENTI', 'CHOA CHU KANG', 'BUKIT PANJANG']
Loading data...

---------BASIC STORE---------
             Column          Pages
              month            444
               town            950
          flat_type           1013
              block            317
        street_name           1271
       storey_range            761
     floor_area_sqm            254
         flat_model           1394
 lease_commence_date            127
       resale_price            254
              Total           6785

ScanResult written to ScanResult_U2222312E.csv (568 records)
```

As shown in the screenshot, the program successfully parsed the matriculation number, loaded the dataset, built the basic and compressed column stores, and completed all queries. A total of 568 valid (x,y) pairs were identified, meaning that all 568 combinations of $x \in \{1, ..., 8\}$ and $y \in \{80, ..., 150\}$ produced at least one matching record with a minimum price per square meter not exceeding 4725 SGD/sqm.

Correctness Verification

To verify the output, the full result was validated programmatically using an independent plain Python reference implementation provided in the project's test suite. The following command was executed:

TEST_MATRIC=U2222312E pytest -v -s tests/test_result.py::TestGroundTruth

```
(sc4023) potalaboonkitrungpaisarn@Potalas-MacBook-Air SC4023-Project-main-main % TEST_MATRIC=U2222312E pytest -v -s tests/test_result.py::TestGroundTruth
============================================================================ test session starts ============================================================================
platform darwin -- Python 3.12.11, pytest-9.0.3, pluggy-1.6.0 -- /Users/potalaboonkitrungpaisarn/opt/anaconda3/envs/sc4023/bin/python3.12
cachedir: .pytest_cache
rootdir: /Users/potalaboonkitrungpaisarn/Downloads/SC4023-Project-main-main
collected 1 item

tests/test_result.py::TestGroundTruth::test_matches_ground_truth
=== Ground truth vs column store (first 5 / last 5 of 568 rows) ===
    src   (x,y)     Year   Month   Town            Block    Area       Flat_Model            Lease   Price/sqm
  ------------------------------------------------------------------------------------------------------------------
    GT[  1]  (1, 80)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    CS[  1]  (1, 80)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    GT[  2]  (1, 81)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    CS[  2]  (1, 81)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    GT[  3]  (1, 82)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    CS[  3]  (1, 82)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    GT[  4]  (1, 83)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    CS[  4]  (1, 83)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    GT[  5]  (1, 84)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
    CS[  5]  (1, 84)   2022   01      BUKIT PANJANG          449      102        Model A               1997         3608
  ------------------------------------------------------------------------------------------------------------------
    GT[564]  (8, 146)  2022   01      CHOA CHU KANG          274      148        Apartment             1993         4257
    CS[564]  (8, 146)  2022   01      CHOA CHU KANG          274      148        Apartment             1993         4257
    GT[565]  (8, 147)  2022   01      CHOA CHU KANG          274      148        Apartment             1993         4257
    CS[565]  (8, 147)  2022   01      CHOA CHU KANG          274      148        Apartment             1993         4257
    GT[566]  (8, 148)  2022   01      CHOA CHU KANG          274      148        Apartment             1993         4257
    CS[566]  (8, 148)  2022   01      CHOA CHU KANG          274      148        Apartment             1993         4257
    GT[567]  (8, 149)  2022   01      CHOA CHU KANG          108      154        Maisonette            1989         4351
    CS[567]  (8, 149)  2022   01      CHOA CHU KANG          108      154        Maisonette            1989         4351
    GT[568]  (8, 150)  2022   01      CHOA CHU KANG          108      154        Maisonette            1989         4351
    CS[568]  (8, 150)  2022   01      CHOA CHU KANG          108      154        Maisonette            1989         4351
PASSED

============================================================================= 1 passed in 300.79s (0:05:00) =============================================================================
```

This test independently recomputes all (x, y) query results using a Python implementation without any column store optimisations, then compares them row-by-row against the column store output. As shown in the screenshot, all 568 (x, y) pairs matched exactly between the ground truth (GT) and column store (CS) outputs, with the test completing in 300 seconds and returning PASSED. This confirms that the column store correctly filters by month range, town, and floor area, and accurately identifies the record with the minimum price per square meter for all query combinations.

Appendix

```python
# — Filtering ————————————————————————————————————

def filter_by_month(self, positions, start_month: int, month_span: int,
                    use_index: bool = False) -> list[int]:
    """Filters records within [start_month, start_month + month_span - 1]."""
    end_month = start_month + month_span - 1
    result = []
    for pos in positions:
        if use_index:
            if not self.store.pos_has_month_in_range(pos, start_month, end_month):
                continue
        month = self.store.get_month(pos)
        if start_month <= month <= end_month:
            result.append(pos)
    return result

def filter_by_town(self, positions: list[int], town: int,
                   use_zone_map: bool = False) -> list[int]:
    result = []
    for pos in positions:
        if use_zone_map:
            zval = self.store.get_town_zone_map_entry(pos)
            if not (zval & (1 << town)):
                continue
        town_val = self.store.get_town(pos)
        if town_val == town:
            result.append(pos)
    return result

def filter_by_area(self, positions: list[int], min_area: int,
                   use_zone_map: bool = False) -> list[int]:
    result = []
    for pos in positions:
        if use_zone_map:
            _, zmax = self.store.get_area_zone_map_entry(pos)
            if zmax < min_area:
                continue
        sqm = self.store.get_floor_area_sqm(pos)
        if sqm >= min_area:
            result.append(pos)
    return result
```

Figure 1: filter_by_month, filter_by_town, filter_by_area functions

```python
def apply_filters(self, start_month: int, town: int,
                  start_idx: int, stop_idx: int,
                  month_span: int = 2, min_area: int = 80) -> list[int]:
    """Composite filter: month range -> town -> area."""
    pos = self.filter_by_month(range(start_idx, stop_idx), start_month, month_span, True)
    pos = self.filter_by_town(pos, town, True)
    pos = self.filter_by_area(pos, min_area, True)
    return pos
```

Figure 2: apply_filters function

```python
def vector_a_time(self, start_month: int, town: int,
                  month_span: int = 2, min_area: int = 80,
                  vector_size: int = 32) -> None:
    """Processes data in chunks for memory-efficient aggregation."""
    self.store.clear_read_state()
    store_size = self.store.get_size()

    min_price = math.inf
    price_sum, price_count = 0, 0
    min_ppsqm = math.inf
    sd_count, sd_avg, sd_ssd = 0, 0, 0

    for vec_start in range(0, store_size, vector_size):
        vec_stop = min(vec_start + vector_size, store_size)

        batch_min = self.minimum_price(start_month, town, vec_start, vec_stop, month_span, min_area)
        batch_sum, batch_cnt = self.average_price(start_month, town, vec_start, vec_stop, month_span, min_area)
        batch_sd_cnt, batch_sd_avg, batch_sd_ssd = self.stddev_price(
            start_month, town, vec_start, vec_stop, month_span, min_area
        )
        batch_min_ppsqm = self.minimum_price_per_sqm(
            start_month, town, vec_start, vec_stop, month_span, min_area
        )

        min_price = min(min_price, batch_min)
        price_sum += batch_sum
        price_count += batch_cnt

        if batch_sd_cnt > 0:
            delta = batch_sd_avg - sd_avg
            new_count = sd_count + batch_sd_cnt
            sd_avg += delta * batch_sd_cnt / new_count
            sd_ssd += batch_sd_ssd + delta * delta * sd_count * batch_sd_cnt / new_count
            sd_count = new_count

        min_ppsqm = min(min_ppsqm, batch_min_ppsqm)

    for metric, val, fallback in [
        (Metrics.MIN_PRICE, min_price, math.inf),
        (Metrics.AVG_PRICE, price_sum / price_count if price_count else None, None),
        (Metrics.STDDEV, math.sqrt(sd_ssd / sd_count) if sd_count else None, None),
        (Metrics.MIN_PRICE_PER_SQM, min_ppsqm, math.inf),
    ]:
        if val is None or val == fallback:
            self._add_result(start_month, town, metric, "No result")
        else:
            self._add_result(start_month, town, metric, val)
```

Figure 3: vector_a_time function

```python
def generate_scan_result(store: DiskColumnStore, matric: str,
                         start_month: int, town_names: list[str]):
    """
    Generates ScanResult_<Matric>.csv by iterating all (x, y) pairs.
    x = month span [1..8], y = min area [80..150].
    For each valid (x, y), finds the record with minimum price/sqm <= 4725.
    """
    town_enc = store.encoders[1]
    towns_encoded = [town_enc.encode(t) for t in town_names]
    engine = ScanEngine(store=store)

    # x - month span (how many months to look back)
    # y - minimum floor area in sqm
    rows = []
    for x in range(1, 9):
        for y in range(80, 151):
            best_pos = None
            best_ppsqm = math.inf

            for town_idx in towns_encoded:
                store.clear_read_state()
                positions = engine.apply_filters(
                    start_month, town_idx, 0, store.get_size(),
                    month_span=x, min_area=y,
                )
                if not positions:
                    continue

                pos, ppsqm = engine.find_min_price_per_sqm_record(positions)
                if ppsqm < best_ppsqm:
                    best_ppsqm = ppsqm
                    best_pos = pos

            if best_pos is not None and best_ppsqm <= 4725:
                rec = engine.get_record_details(best_pos)
                rows.append([
                    f"({x}, {y})",
                    rec["year"],
                    rec["month"],
                    rec["town"],
                    rec["block"],
                    rec["floor_area"],
                    rec["flat_model"],
                    rec["lease_commence_date"],
                    round(best_ppsqm),
                ])

    output_path = f"ScanResult_{matric}.csv"
    with open(output_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([
            "(x, y)", "Year", "Month", "Town", "Block",
            "Floor_Area", "Flat_Model", "Lease_Commence_Date",
            "Price_Per_Square_Meter",
        ])
        if rows:
            writer.writerows(rows)
        else:
            writer.writerow(["No result", "", "", "", "", "", "", "", ""])

    print(f"\nScanResult written to {output_path} ({len(rows)} records)")
```

Figure 4: generate_scan_result function

```python
def get_record_details(self, pos: int) -> dict:
    """Retrieves full record details for ScanResult CSV output."""
    month_val = self.store.get_month(pos)
    month_str = self.store.decode_month(month_val)
    month_name, year_suffix = month_str.split("-")
    year = f"20{year_suffix}"
    month_num = str(MONTH_NUM[month_name]).zfill(2)

    return {
        "year": year,
        "month": month_num,
        "town": self.store.decode_town(self.store.get_town(pos)),
        "block": self.store.decode_block_id(self.store.get_block_id(pos)),
        "floor_area": int(self.store.get_floor_area_sqm(pos)),
        "flat_model": self.store.decode_flat_model(self.store.get_flat_model(pos)),
        "lease_commence_date": int(self.store.get_lease_date(pos)),
    }
```

Figure 5: get_record_details function

```python
def get_item(self, pos: int, i: int):
    """Retrieves the decoded value from column i at position pos, loading the page if needed."""
    page_number = self._pos_to_page(pos, i)
    if self.read_pointers[i] is None:
        self.read_pointers[i] = open(self.columns[i], "rb")

    # Only read from disk if the requested page is not already buffered.
    # tell() == (page+1)*PAGE_SIZE means we just finished reading that page.
    if self.read_pointers[i].tell() != (page_number + 1) * PAGE_SIZE:
        self.reads += 1
        self.read_pointers[i].seek(page_number * PAGE_SIZE)
        self.read_buffers[i] = self.read_pointers[i].read(PAGE_SIZE)

    byte_width = self.encoders[i].byte_width()
    items_per_page = PAGE_SIZE // byte_width
    start = (pos % items_per_page) * byte_width
    return self.encoders[i].deserialize(self.read_buffers[i][start : start + byte_width])
```

Figure 6: get_item function